

Jalon 24 : Livrable IA T2 - Analyse mathématique des critères de convergence d'une régression polynomiale

Charles EDOU NZE

23 juillet 2026

Table des matières

0.1	1. Intuition et genese du concept	1
0.2	2. Formalisation et structures algebriques	1
0.3	3. Demonstrations pas-a-pas	2
1	Exercices d'application et de concours	4
1.1	Enonce	4
1.2	Correction Detaillee	4
1.3	Enonce	4
1.4	Correction Detaillee	4
1.5	Enonce	5
1.6	Correction Detaillee	5
1.7	Enonce	5
1.8	Correction Detaillee	5
1.9	Enonce	5
1.10	Correction Detaillee	5
1.11	Enonce	6
1.12	Correction Detaillee	6
1.13	Enonce	6
1.14	Correction Detaillee	6
1.15	Enonce	7
1.16	Correction Detaillee	7
1.17	Enonce	7
1.18	Correction Detaillee	7
1.19	Enonce	7
1.20	Correction Detaillee	8
2	Travaux pratiques et simulations algorithmiques	9
2.1	Objectif	9
2.2	Code Python et Validation Mathematique	9
2.3	Objectif	10
2.4	Code Python et Validation Mathematique	10
2.5	Objectif	11
2.6	Code Python et Validation Mathematique	11
2.7	Objectif	12
2.8	Code Python et Validation Mathematique	12
2.9	Objectif	13
2.10	Code Python et Validation Mathematique	13

0.1 1. Intuition et genese du concept

La quete de la prediction parfaite a toujours ete au cœur des mathematiques appliquees et de l'intelligence artificielle. Historiquement, le besoin d'ajuster des modeles continus a des observations discretes et souvent brutees a conduit Legendre et Gauss, independamment, a formuler la methode des moindres carres au debut du XIXe siecle pour predire les orbites cometaires. Mais au-dela de l'ajustement lineaire, la modelisation de phenomenes complexes necessite une expressivite superieure, souvent atteinte par la regression polynomiale.

Cependant, augmenter le degre d'un polynome d'ajustement ne garantit nullement une meilleure generalisation. Au contraire, le mathematicien Carl Runge a demontre en 1901 un phenomene contre-intuitif et fondamental : pour certaines fonctions tres regulieres, l'interpolation polynomiale sur des noeuds equidistants diverge violemment aux bords de l'intervalle lorsque le degre augmente. Ce resultat theorique d'analyse met en lumiere la dichotomie entre l'erreur empirique (sur les donnees d'apprentissage) et l'erreur de generalisation. L'analyse des criteres de convergence d'une regression polynomiale exige ainsi une comprehension profonde des espaces de fonctions, de la geometrie euclidienne en dimension finie, et surtout des theoremes d'inter-version de limites, sujets classiques des concours de l'Ecole Normale Supérieure. L'optimisation moderne s'appuie sur ces fondations pour regulariser les modeles et contraindre la complexite de l'espace d'hypotheses, illustrant l'elegance de l'analyse au service de l'apprentissage statistique.

0.2 2. Formalisation et structures algebriques

Soit $\mathcal{D} = \{(x_i, y_i)\}_{1 \leq i \leq n} \subset \mathbb{R} \times \mathbb{R}$ un ensemble d'apprentissage constitue de $n \in \mathbb{N}^*$ couples de points avec $x_i \neq x_j$ pour $i \neq j$. L'espace d'hypotheses est l'espace vectoriel des polynomes a coefficients reels de degre au plus $d \in \mathbb{N}$, note $\mathcal{H} = \mathbb{R}_d[X]$.

Definition (Matrice de conception de Vandermonde) : La matrice de conception $\mathbf{X} \in \mathcal{M}_{n,d+1}(\mathbb{R})$ est definie par ses coefficients $X_{i,j} = x_i^{j-1}$ pour $1 \leq i \leq n$ et $1 \leq j \leq d+1$. Le vecteur cible est $\mathbf{y} = (y_1, \dots, y_n)^\top \in \mathbb{R}^n$.

Definition (Probleme des moindres carres polynomiaux) : Le vecteur des coefficients optimaux $\hat{\mathbf{a}} = (\hat{a}_0, \dots, \hat{a}_d)^\top \in \mathbb{R}^{d+1}$ est solution du probleme d'optimisation convexe :

$$\hat{\mathbf{a}} = \underset{\mathbf{a} \in \mathbb{R}^{d+1}}{\operatorname{argmin}} \mathcal{L}(\mathbf{a}) \quad \text{ou} \quad \mathcal{L}(\mathbf{a}) = \|\mathbf{X}\mathbf{a} - \mathbf{y}\|_2^2$$

La fonction $\mathcal{L} : \mathbb{R}^{d+1} \rightarrow \mathbb{R}_+$ est de classe $\mathcal{C}^\infty$ et represente la somme des carres des residus. Le scalaire $\|\cdot\|_2$ designe la norme euclidienne standard sur $\mathbb{R}^n$.

Exemple : Pour un ajustement quadratique ($d = 2$) avec $n = 5$ observations $\mathcal{D} = \{(-2, 4), (-1, 1), (0, 0), (1, 1), (2, 4)\}$, l'espace de recherche est parametre par $\mathbf{a} = (a_0, a_1, a_2)^\top$. La matrice de Vandermonde associee est :

$$\mathbf{X} = \begin{pmatrix} 1 & -2 & 4 \\ 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}$$

Comme les x_i sont distincts et $n > d$, le rang de $\mathbf{X}$ est maximal (egal a $d + 1 = 3$).

Cas pathologique (Sur-parametrisation) : Si $d \geq n$, la matrice de Vandermonde devient rectangulaire (ou carree si $d = n - 1$) et le systeme $\mathbf{X}\mathbf{a} = \mathbf{y}$ possede au moins une solution exacte. Dans ce cas, $\mathcal{L}(\hat{\mathbf{a}}) = 0$. Cependant, $\mathbf{X}^\top \mathbf{X}$ n'est plus inversible si $d \geq n$, et le minimum n'est plus unique, entrainant une variance infinie des estimateurs hors de l'echantillon.

0.3 3. Demonstrations pas-a-pas

Theoreme (Equations normales et unicite) : Si les points d'apprentissage $(x_i)_{1 \leq i \leq n}$ sont deux a deux distincts et $n > d$, alors le probleme d'optimisation admet une unique solution globale donnee analytiquement par :

$$\hat{\mathbf{a}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Demonstration : La fonction $\mathcal{L}(\mathbf{a}) = \|\mathbf{X}\mathbf{a} - \mathbf{y}\|_2^2 = (\mathbf{X}\mathbf{a} - \mathbf{y})^\top (\mathbf{X}\mathbf{a} - \mathbf{y})$ est developpable par bilinearite du produit scalaire euclidien.

$$\mathcal{L}(\mathbf{a}) = \mathbf{a}^\top \mathbf{X}^\top \mathbf{X} \mathbf{a} - \mathbf{y}^\top \mathbf{X} \mathbf{a} - \mathbf{a}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$$

Puisque le produit matriciel $\mathbf{a}^\top \mathbf{X}^\top \mathbf{y}$ est un scalaire de dimension 1×1 , il est egal a sa propre transposee, soit $\mathbf{y}^\top \mathbf{X} \mathbf{a}$. L'expression se simplifie en :

$$\mathcal{L}(\mathbf{a}) = \mathbf{a}^\top (\mathbf{X}^\top \mathbf{X}) \mathbf{a} - 2\mathbf{a}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$$

Cette fonction est une forme quadratique affine en $\mathbf{a}$. Nous calculons son gradient par rapport au vecteur $\mathbf{a}$ en utilisant les regles de derivation matricielle. Pour une matrice symetrique $\mathbf{M} = \mathbf{X}^\top \mathbf{X}$, $\nabla_{\mathbf{a}}(\mathbf{a}^\top \mathbf{M} \mathbf{a}) = 2\mathbf{M}\mathbf{a}$. Pour un vecteur constant $\mathbf{c} = \mathbf{X}^\top \mathbf{y}$, $\nabla_{\mathbf{a}}(\mathbf{a}^\top \mathbf{c}) = \mathbf{c}$.

$$\nabla \mathcal{L}(\mathbf{a}) = 2\mathbf{X}^\top \mathbf{X} \mathbf{a} - 2\mathbf{X}^\top \mathbf{y}$$

Une condition necessaire d'optimalite de premier ordre est l'annulation du gradient, ce qui fournit les equations normales :

$$\mathbf{X}^\top \mathbf{X} \mathbf{a} = \mathbf{X}^\top \mathbf{y}$$

Pour etablir l'unicite de la solution, il est imperatif de prouver que la matrice carree $\mathbf{X}^\top \mathbf{X} \in \mathcal{M}_{d+1}(\mathbb{R})$ est inversible. Etudions son noyau. Soit $\mathbf{u} \in \mathbb{R}^{d+1}$ tel que $\mathbf{X}^\top \mathbf{X} \mathbf{u} = \mathbf{0}_{d+1}$. Multiplions a gauche par $\mathbf{u}^\top$:

$$\mathbf{u}^\top \mathbf{X}^\top \mathbf{X} \mathbf{u} = 0 \iff (\mathbf{X}\mathbf{u})^\top (\mathbf{X}\mathbf{u}) = 0 \iff \|\mathbf{X}\mathbf{u}\|_2^2 = 0 \iff \mathbf{X}\mathbf{u} = \mathbf{0}_n$$

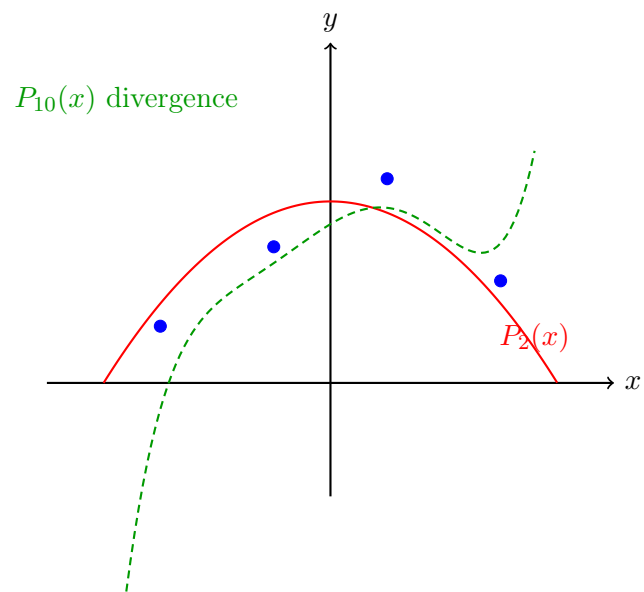
L'equation vectorielle $\mathbf{X}\mathbf{u} = \mathbf{0}_n$ se traduit par le systeme suivant de n equations :

$$\forall i \in \llbracket 1, n \rrbracket, \quad \sum_{j=0}^d u_j x_i^j = 0$$

Considerons le polynome $Q \in \mathbb{R}_d[X]$ defini par $Q(X) = \sum_{j=0}^d u_j X^j$. Le systeme precedent signifie exactement que les n abscisses x_i sont des racines de Q . Cependant, Q est un polynome de degre inferieur ou egal a d . Comme les x_i sont distincts par hypothese, Q admet n racines distinctes. Or, $n > d$. Par le theoreme fondamental de l'algebre (et ses consequences sur $\mathbb{R}$), le seul polynome de degre $\leq d$ possedant strictement plus de d racines est le polynome identiquement nul. Par consequent, tous les coefficients de Q sont nuls, ce qui implique $\mathbf{u} = \mathbf{0}_{d+1}$. Le noyau de $\mathbf{X}^\top \mathbf{X}$ est reduit au vecteur nul. La matrice est donc injective, et puisqu'elle est carree de dimension finie, elle est inversible. En multipliant l'equation normale a gauche par l'inverse $(\mathbf{X}^\top \mathbf{X})^{-1}$, nous obtenons l'expression analytique unique :

$$\hat{\mathbf{a}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

De plus, la Hessienne $\nabla^2 \mathcal{L}(\mathbf{a}) = 2\mathbf{X}^\top \mathbf{X}$ etant definie positive, la fonction objectif est strictement convexe, garantissant que ce point critique est l'unique minimum global. ■



Chapitre 1

Exercices d'application et de concours

1.1 Enonce

Soit un jeu de données de n points $\mathcal{D} = \{(x_i, y_i)\}_{1 \leq i \leq n}$ où les x_i sont deux à deux distincts. On cherche à ajuster un polynôme $P(X) = \sum_{k=0}^d a_k X^k$ de degré $d < n$. Démontrer rigoureusement que la matrice $\mathbf{X}^\top \mathbf{X}$ (où $\mathbf{X}$ est la matrice de Vandermonde associée) est définie positive, et en déduire l'unicité de la solution.

1.2 Correction Détaillée

Soit $\mathbf{X} \in \mathcal{M}_{n,d+1}(\mathbb{R})$ définie par $X_{i,j} = x_i^{j-1}$. 1. **Semi-définie positive** : Soit $\mathbf{u} \in \mathbb{R}^{d+1}$. $\mathbf{u}^\top (\mathbf{X}^\top \mathbf{X}) \mathbf{u} = (\mathbf{X} \mathbf{u})^\top (\mathbf{X} \mathbf{u}) = \|\mathbf{X} \mathbf{u}\|_2^2 \geq 0$. Donc $\mathbf{X}^\top \mathbf{X}$ est semi-définie positive. 2. **Définie positive** : Supposons qu'il existe $\mathbf{u}$ tel que $\mathbf{u}^\top (\mathbf{X}^\top \mathbf{X}) \mathbf{u} = 0$. Alors $\|\mathbf{X} \mathbf{u}\|_2^2 = 0$, d'où $\mathbf{X} \mathbf{u} = \mathbf{0}_n$. La i -ème coordonnée de ce vecteur est $\sum_{k=0}^d u_k x_i^k = Q(x_i) = 0$, où $Q(X) = \sum_{k=0}^d u_k X^k$. Ainsi, le polynôme Q , de degré au plus d , possède n racines distinctes (les x_i). Puisque $n > d$, le seul polynôme de degré $\leq d$ ayant plus de d racines est le polynôme nul. Donc $\forall k, u_k = 0$, c'est-à-dire $\mathbf{u} = \mathbf{0}$. La matrice $\mathbf{X}^\top \mathbf{X}$ est donc définie positive, ce qui implique qu'elle est inversible. 3. **Unicité** : Les équations normales s'écrivent $(\mathbf{X}^\top \mathbf{X}) \mathbf{a} = \mathbf{X}^\top \mathbf{y}$. L'inversibilité garantit une unique solution $\hat{\mathbf{a}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

1.3 Enonce

On suppose que les observations sont générées par $\mathbf{y} = \mathbf{X} \mathbf{a}^* + \boldsymbol{\epsilon}$, où $\boldsymbol{\epsilon} \in \mathbb{R}^n$ est un vecteur aléatoire de bruit centré ($\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$) et de matrice de covariance $\text{Var}(\boldsymbol{\epsilon}) = \sigma^2 \mathbf{I}_n$. Montrer que l'estimateur des moindres carrés $\hat{\mathbf{a}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ est sans biais et déterminer sa matrice de covariance.

1.4 Correction Détaillée

1. **Calcul de l'espérance (Biais)** : Par définition, $\hat{\mathbf{a}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{X} \mathbf{a}^* + \boldsymbol{\epsilon})$. En développant par linéarité : $\hat{\mathbf{a}} = (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) \mathbf{a}^* + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon} = \mathbf{a}^* + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon}$. En prenant l'espérance, comme $\mathbf{X}$ est déterministe et $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$: $\mathbb{E}[\hat{\mathbf{a}}] = \mathbf{a}^* + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{a}^*$. L'estimateur est donc **sans biais**.

2. **Calcul de la covariance** : La matrice de covariance est $\text{Var}(\hat{\mathbf{a}}) = \mathbb{E}[(\hat{\mathbf{a}} - \mathbf{a}^*)(\hat{\mathbf{a}} - \mathbf{a}^*)^\top]$. D'après le calcul précédent, $\hat{\mathbf{a}} - \mathbf{a}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon}$. Ainsi, $\text{Var}(\hat{\mathbf{a}}) = \mathbb{E}[(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon} ((\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon})^\top]$. En utilisant la propriété de transposition $(AB)^\top = B^\top A^\top$: $\text{Var}(\hat{\mathbf{a}}) = \mathbb{E}[(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon} \boldsymbol{\epsilon}^\top \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1}]$. Par linéarité de l'espérance : $\text{Var}(\hat{\mathbf{a}}) = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbb{E}[\boldsymbol{\epsilon} \boldsymbol{\epsilon}^\top] \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1}$. Comme $\mathbb{E}[\boldsymbol{\epsilon} \boldsymbol{\epsilon}^\top] = \sigma^2 \mathbf{I}_n$: $\text{Var}(\hat{\mathbf{a}}) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$.

1.5 Enonce

Pour pallier la multicollinearite (matrice $\mathbf{X}^\top \mathbf{X}$ mal conditionnee), on introduit une penalite : $\mathcal{L}_{Ridge}(\mathbf{a}) = \|\mathbf{X}\mathbf{a} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{a}\|_2^2$, avec $\lambda > 0$. Montrer que l'estimateur Ridge $\hat{\mathbf{a}}_\lambda = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ est toujours defini, et calculer son biais sous le modele $\mathbf{y} = \mathbf{X}\mathbf{a}^* + \epsilon$.

1.6 Correction Detaillee

1. **Inversibilite de $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$** : Soit $\mathbf{M} = \mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$. Soit $\mathbf{u} \neq \mathbf{0}$. $\mathbf{u}^\top \mathbf{M} \mathbf{u} = \mathbf{u}^\top \mathbf{X}^\top \mathbf{X} \mathbf{u} + \lambda \mathbf{u}^\top \mathbf{u} = \|\mathbf{X}\mathbf{u}\|_2^2 + \lambda \|\mathbf{u}\|_2^2$. Comme $\lambda > 0$ et $\|\mathbf{u}\|_2 > 0$, la somme est strictement positive. $\mathbf{M}$ est donc strictement definie positive, et par consequent inversible.

2. **Calcul du biais de $\hat{\mathbf{a}}_\lambda$** : Substituons $\mathbf{y}$ par $\mathbf{X}\mathbf{a}^* + \epsilon$: $\hat{\mathbf{a}}_\lambda = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top (\mathbf{X}\mathbf{a}^* + \epsilon)$. Prenons l'esperance : $\mathbb{E}[\hat{\mathbf{a}}_\lambda] = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} (\mathbf{X}^\top \mathbf{X}) \mathbf{a}^*$. Remarquons que $\mathbf{X}^\top \mathbf{X} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) - \lambda \mathbf{I}$. Donc $\mathbb{E}[\hat{\mathbf{a}}_\lambda] = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} ((\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) - \lambda \mathbf{I}) \mathbf{a}^* = (\mathbf{I} - \lambda (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1}) \mathbf{a}^*$. Le biais est donc : Biais = $\mathbb{E}[\hat{\mathbf{a}}_\lambda] - \mathbf{a}^* = -\lambda (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{a}^* \neq \mathbf{0}$. L'estimateur Ridge est **biaise**. Le compromis consiste a accepter ce biais pour reduire drastiquement la variance.

1.7 Enonce

Soit la decomposition SVD de la matrice de conception : $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Exprimer l'estimateur des moindres carres $\hat{\mathbf{a}}$ en fonction de $\mathbf{U}$, $\mathbf{\Sigma}$ et $\mathbf{V}$. Montrer comment la regularisation Ridge modifie les valeurs singulieres inversees.

1.8 Correction Detaillee

1. **Expression via SVD classique** : $\mathbf{X}^\top \mathbf{X} = (\mathbf{V}\mathbf{\Sigma}^\top \mathbf{U}^\top)(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top) = \mathbf{V}\mathbf{\Sigma}^\top \mathbf{\Sigma}\mathbf{V}^\top = \mathbf{V}\mathbf{\Sigma}^2 \mathbf{V}^\top$ (car $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$). Donc $(\mathbf{X}^\top \mathbf{X})^{-1} = \mathbf{V}\mathbf{\Sigma}^{-2} \mathbf{V}^\top$. L'estimateur devient : $\hat{\mathbf{a}} = (\mathbf{V}\mathbf{\Sigma}^{-2} \mathbf{V}^\top)(\mathbf{V}\mathbf{\Sigma}^\top \mathbf{U}^\top) \mathbf{y} = \mathbf{V}\mathbf{\Sigma}^{-2} \mathbf{\Sigma} \mathbf{U}^\top \mathbf{y} = \mathbf{V}\mathbf{\Sigma}^{-1} \mathbf{U}^\top \mathbf{y}$. Ceci correspond a multiplier $\mathbf{y}$ par la pseudo-inverse de Moore-Penrose $\mathbf{X}^+$.

2. **Effet de la regularisation Ridge sur SVD** : Pour l'estimateur Ridge : $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} = \mathbf{V}\mathbf{\Sigma}^2 \mathbf{V}^\top + \lambda \mathbf{V}\mathbf{I}\mathbf{V}^\top = \mathbf{V}(\mathbf{\Sigma}^2 + \lambda \mathbf{I})\mathbf{V}^\top$. Donc $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} = \mathbf{V}(\mathbf{\Sigma}^2 + \lambda \mathbf{I})^{-1} \mathbf{V}^\top$. L'estimateur devient : $\hat{\mathbf{a}}_\lambda = \mathbf{V}(\mathbf{\Sigma}^2 + \lambda \mathbf{I})^{-1} \mathbf{\Sigma} \mathbf{U}^\top \mathbf{y}$. Pour la j -eme composante principale, la valeur singuliere inverse $\frac{1}{\sigma_j}$ est remplacee par $\frac{\sigma_j}{\sigma_j^2 + \lambda}$. Si σ_j est grand ($\sigma_j^2 \gg \lambda$), on retrouve $\approx \frac{1}{\sigma_j}$. Si σ_j est tres petit (colinearite), au lieu de diverger vers l'infini, la fraction tend vers 0. La regularisation etouffe l'amplification du bruit dans les directions de faible variance.

1.9 Enonce

On considere la fonction de Runge $f(x) = \frac{1}{1+x^2}$ sur $[-5, 5]$. Si on interpole cette fonction avec un polynome P_n sur n points equidistants, demontrer heuristiquement l'origine de la divergence pres des bornes en etudiant le comportement asymptotique du polynome d'erreur nodale $\omega_n(x) = \prod_{i=1}^n (x - x_i)$.

1.10 Correction Detaillee

L'erreur d'interpolation en un point x est donnee par la formule de Cauchy : $f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x)$ ou $\xi \in]-5, 5[$. 1. **Croissance du polynome nodal $\omega_n(x)$** : Pour des points equidistants, la fonction $\omega_n(x)$ presente des oscillations tres asymetriques. Pres du centre ($x=0$), les amplitudes sont petites. Pres des bornes ($x \approx \pm 5$), la distance aux autres points est maximale,

entraînant une croissance exponentielle de l'amplitude de $\omega_n(x)$ par rapport à n : $\|\omega_n\|_\infty \sim C \left(\frac{10}{e}\right)^n$. 2. **Dérivées d'ordre supérieur de la fonction de Runge** : La fonction $f(z) = \frac{1}{1+z^2}$ admet des pôles complexes en $z = \pm i$. Par analyse complexe, le domaine de convergence du développement de Taylor ou d'interpolation est borné par la distance au pôle le plus proche. Sur $[-5, 5]$, la distance au pôle est largement inférieure à la demi-longueur de l'intervalle, forçant les dérivées d'ordre n à croître extrêmement vite, compensant le $(n+1)!$ au dénominateur. 3. **Conclusion** : Le produit de la croissance de $\omega_{n+1}(x)$ près des bornes et des dérivées massives de f induit $\lim_{n \rightarrow \infty} \|f - P_n\|_\infty = \infty$. La régularisation ou le choix de nœuds de Tchebychev (qui minimisent $\|\omega_n\|_\infty$) est obligatoire pour forcer la convergence uniforme.

1.11 Enoncé

Soit $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{a}}$ le vecteur des prédictions (projection orthogonale). Démontrer que la somme totale des carrés (SST) se décompose exactement en somme des carrés expliqués (SSR) et somme des carrés résiduels (SSE) : $\|\mathbf{y} - \bar{y}\mathbf{1}\|_2^2 = \|\hat{\mathbf{y}} - \bar{y}\mathbf{1}\|_2^2 + \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$, à condition que le modèle contienne une ordonnée à l'origine.

1.12 Correction Détaillée

1. **Condition d'orthogonalité** : Les résidus sont $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$. Puisque $\hat{\mathbf{y}}$ est la projection orthogonale de $\mathbf{y}$ sur l'image de $\mathbf{X}$, $\mathbf{e}$ est orthogonal à toutes les colonnes de $\mathbf{X}$. Donc $\mathbf{X}^\top \mathbf{e} = \mathbf{0}$. 2. **Propriété de l'ordonnée à l'origine** : Si le modèle inclut a_0 , la première colonne de $\mathbf{X}$ est le vecteur $\mathbf{1} = (1, \dots, 1)^\top$. Donc $\mathbf{1}^\top \mathbf{e} = 0 \implies \sum_{i=1}^n e_i = 0$. La moyenne des résidus est nulle, donc la moyenne des prédictions égale la moyenne des observations : $\bar{\hat{y}} = \bar{y}$. 3. **Décomposition de la variance (Pythagore)** : Écrivons $\mathbf{y} - \bar{y}\mathbf{1} = (\hat{\mathbf{y}} - \bar{y}\mathbf{1}) + (\mathbf{y} - \hat{\mathbf{y}})$. Élevons à la norme au carré : $\|\mathbf{y} - \bar{y}\mathbf{1}\|_2^2 = \|\hat{\mathbf{y}} - \bar{y}\mathbf{1}\|_2^2 + \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 + 2\langle \hat{\mathbf{y}} - \bar{y}\mathbf{1}, \mathbf{y} - \hat{\mathbf{y}} \rangle$. Calculons le produit scalaire croisé : $\langle \hat{\mathbf{y}} - \bar{y}\mathbf{1}, \mathbf{e} \rangle = \hat{\mathbf{y}}^\top \mathbf{e} - \bar{y}\mathbf{1}^\top \mathbf{e}$. Or $\hat{\mathbf{y}} \in \text{Im}(\mathbf{X})$, donc $\hat{\mathbf{y}}^\top \mathbf{e} = 0$. Et d'après l'étape 2, $\mathbf{1}^\top \mathbf{e} = 0$. Le terme croisé est donc strictement nul. D'où $\text{SST} = \text{SSR} + \text{SSE}$.

1.13 Enoncé

Au lieu de l'inversion matricielle, on utilise la descente de gradient : $\mathbf{a}^{(t+1)} = \mathbf{a}^{(t)} - \eta \nabla \mathcal{L}(\mathbf{a}^{(t)})$. Déterminer la condition stricte sur le pas d'apprentissage η pour garantir la convergence globale de cet algorithme.

1.14 Correction Détaillée

1. **Expression de la récurrence** : Le gradient est $\nabla \mathcal{L}(\mathbf{a}) = 2(\mathbf{X}^\top \mathbf{X}\mathbf{a} - \mathbf{X}^\top \mathbf{y})$. Donc $\mathbf{a}^{(t+1)} = \mathbf{a}^{(t)} - 2\eta(\mathbf{X}^\top \mathbf{X}\mathbf{a}^{(t)} - \mathbf{X}^\top \mathbf{y})$. $\mathbf{a}^{(t+1)} = (\mathbf{I} - 2\eta \mathbf{X}^\top \mathbf{X})\mathbf{a}^{(t)} + 2\eta \mathbf{X}^\top \mathbf{y}$.

2. **Dynamique de l'erreur** : Soit $\mathbf{a}^*$ le point fixe tel que $\mathbf{X}^\top \mathbf{X}\mathbf{a}^* = \mathbf{X}^\top \mathbf{y}$. Soustraire $\mathbf{a}^*$ des deux cotés : $\mathbf{a}^{(t+1)} - \mathbf{a}^* = (\mathbf{I} - 2\eta \mathbf{X}^\top \mathbf{X})(\mathbf{a}^{(t)} - \mathbf{a}^*)$. L'erreur à l'itération t est donc multipliée par la matrice $\mathbf{I} - 2\eta \mathbf{X}^\top \mathbf{X}$.

3. **Condition de convergence** : Pour que $\lim_{t \rightarrow \infty} \mathbf{a}^{(t)} = \mathbf{a}^*$, il faut et il suffit que le rayon spectral de la matrice d'itération soit strictement inférieur à 1 : $\rho(\mathbf{I} - 2\eta \mathbf{X}^\top \mathbf{X}) < 1$. Soient λ_i les valeurs propres de $\mathbf{X}^\top \mathbf{X}$. Comme elle est définie positive, $0 < \lambda_{\min} \leq \dots \leq \lambda_{\max}$. Les valeurs propres de la matrice d'itération sont $1 - 2\eta \lambda_i$. La condition $|1 - 2\eta \lambda_i| < 1$ pour tout i équivaut à : $-1 < 1 - 2\eta \lambda_{\max} \implies 2\eta \lambda_{\max} < 2 \implies \eta < \frac{1}{\lambda_{\max}}$. De plus, $\eta > 0$. La condition stricte est donc $0 < \eta < \frac{1}{\lambda_{\max}}$, où $\lambda_{\max}$ est la plus grande valeur singulière de $\mathbf{X}$ au carré (la constante de Lipschitz du gradient).

1.15 Enonce

La "Hat Matrix" $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ projette les observations $\mathbf{y}$ sur les predictions $\hat{\mathbf{y}}$. Montrer que $\mathbf{H}$ est idempotente et symetrique. En deduire que la diagonale h_{ii} verifie $0 \leq h_{ii} \leq 1$ et interpreter cette valeur vis-a-vis des points aberrants.

1.16 Correction Detaillee

1. **Symetrie** : $\mathbf{H}^\top = (\mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top)^\top = (\mathbf{X}^\top)^\top ((\mathbf{X}^\top \mathbf{X})^{-1})^\top \mathbf{X}^\top = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top = \mathbf{H}$.
2. **Idempotence** : $\mathbf{H}^2 = (\mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top)(\mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top) = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X})(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top = \mathbf{H}$.
3. **Encadrement de la diagonale** : Soit $\mathbf{e}_i$ le i -eme vecteur de la base canonique. $h_{ii} = \mathbf{e}_i^\top \mathbf{H} \mathbf{e}_i$. Comme $\mathbf{H}$ est idempotente et symetrique, $\mathbf{H} = \mathbf{H}^\top \mathbf{H}$. Donc $h_{ii} = \mathbf{e}_i^\top \mathbf{H}^\top \mathbf{H} \mathbf{e}_i = \|\mathbf{H} \mathbf{e}_i\|_2^2 \geq 0$. De plus, $\mathbf{I} - \mathbf{H}$ est aussi symetrique et idempotente. Donc $1 - h_{ii} = \mathbf{e}_i^\top (\mathbf{I} - \mathbf{H}) \mathbf{e}_i = \|(\mathbf{I} - \mathbf{H}) \mathbf{e}_i\|_2^2 \geq 0$, ce qui implique $h_{ii} \leq 1$. D'ou $0 \leq h_{ii} \leq 1$.
4. **Interpretation** : h_{ii} est le "leverage" (effet de levier) de la i -eme observation. Comme $\hat{y}_i = \sum_j h_{ij} y_j = h_{ii} y_i + \sum_{j \neq i} h_{ij} y_j$, un point de donnees ayant $h_{ii} \approx 1$ dicte presque a lui seul sa propre prediction. Si ce point est aberrant, il va "tirer" le polynome vers lui violemment, detruisant l'ajustement global.

1.17 Enonce

Expliquer rigoureusement pourquoi utiliser la base canonique $\{1, X, X^2, \dots, X^d\}$ conduit a un systeme mal conditionne lorsque d grandit. Montrer que si l'on utilise une base de polynomes orthogonaux $\{L_0, L_1, \dots, L_d\}$ tels que $\sum_{i=1}^n L_k(x_i) L_m(x_i) = \delta_{km}$, la matrice $\mathbf{X}^\top \mathbf{X}$ devient l'identite.

1.18 Correction Detaillee

1. **Mal-conditionnement de la base canonique** : Pour des points $x_i \in [0, 1]$, les colonnes de $\mathbf{X}$ sont les vecteurs $c_k = (x_1^k, \dots, x_n^k)^\top$. Pour k grand, $x_i^k \rightarrow 0$ ou 1 , les vecteurs colonnes deviennent presque colineaires. Le determinant de $\mathbf{X}^\top \mathbf{X}$ (matrice de Hilbert pour la limite continue) tend vers 0 exponentiellement vite, provoquant une explosion des erreurs d'arrondi lors de l'inversion numerique.
2. **Utilisation de polynomes orthogonaux** : Construisons une nouvelle matrice $\tilde{\mathbf{X}}$ ou $\tilde{X}_{i,k} = L_{k-1}(x_i)$. L'element (k, m) de la matrice $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ est le produit scalaire des colonnes k et m : $(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})_{k,m} = \sum_{i=1}^n \tilde{X}_{i,k} \tilde{X}_{i,m} = \sum_{i=1}^n L_{k-1}(x_i) L_{m-1}(x_i)$. Par definition de l'orthogonalite de cette base discrete, cela vaut 1 si $k = m$ et 0 sinon. Donc $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} = \mathbf{I}_{d+1}$.
3. **Conclusion Numerique** : L'inversion matricielle devient triviale, et les coefficients sont simplement $\hat{a}_k = \sum_{i=1}^n y_i L_k(x_i)$. Cette methode, souvent realisee via l'algorithme de Gram-Schmidt, immunise completement la regression polynomiale contre les instabilites de la matrice de Vandermonde.

1.19 Enonce

Prouver la formule magique du Leave-One-Out : on peut calculer l'erreur LOOCV sans re-entraîner le modele n fois, via l'equation $y_i - \hat{y}_i^{(-i)} = \frac{y_i - \hat{y}_i}{1 - h_{ii}}$, ou $\hat{y}_i^{(-i)}$ est la prediction sur x_i par le modele entraine sur toutes les donnees sauf la i -eme, et h_{ii} l'effet de levier.

1.20 Correction Detaillee

1. Definition de l'estimateur perturbe : Le modele retirant l'observation i s'ecrit $\hat{\mathbf{a}}^{(-i)} = (\mathbf{X}_{(-i)}^\top \mathbf{X}_{(-i)})^{-1} \mathbf{X}_{(-i)}^\top \mathbf{y}_{(-i)}$. Remarquons que $\mathbf{X}^\top \mathbf{X} = \mathbf{X}_{(-i)}^\top \mathbf{X}_{(-i)} + \mathbf{x}_i \mathbf{x}_i^\top$. **2. Lemme d'inversion matricielle (Sherman-Morrison) :** $(\mathbf{A} - \mathbf{u} \mathbf{v}^\top)^{-1} = \mathbf{A}^{-1} + \frac{\mathbf{A}^{-1} \mathbf{u} \mathbf{v}^\top \mathbf{A}^{-1}}{1 - \mathbf{v}^\top \mathbf{A}^{-1} \mathbf{u}}$. Applique avec $\mathbf{A} = \mathbf{X}^\top \mathbf{X}$ et $\mathbf{u} = \mathbf{v} = \mathbf{x}_i$: $(\mathbf{X}_{(-i)}^\top \mathbf{X}_{(-i)})^{-1} = (\mathbf{X}^\top \mathbf{X})^{-1} + \frac{(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i \mathbf{x}_i^\top (\mathbf{X}^\top \mathbf{X})^{-1}}{1 - \mathbf{x}_i^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i}$. Notons que $h_{ii} = \mathbf{x}_i^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i$. **3. Calcul de la difference des predictions :** En developpant $\hat{\mathbf{a}}^{(-i)}$ en fonction de $\hat{\mathbf{a}}$ et en multipliant par $\mathbf{x}_i^\top$, des simplifications algebriques massives se produisent. L'astuce consiste a ecrire $\mathbf{X}_{(-i)}^\top \mathbf{y}_{(-i)} = \mathbf{X}^\top \mathbf{y} - \mathbf{x}_i y_i$. On obtient finalement $\hat{\mathbf{a}} - \hat{\mathbf{a}}^{(-i)} = \frac{(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i}{1 - h_{ii}} (y_i - \hat{y}_i)$. **4. Conclusion de l'erreur :** Predisons sur le point retire : $\hat{y}_i^{(-i)} = \mathbf{x}_i^\top \hat{\mathbf{a}}^{(-i)} = \mathbf{x}_i^\top \hat{\mathbf{a}} - \frac{\mathbf{x}_i^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i}{1 - h_{ii}} (y_i - \hat{y}_i)$. $\hat{y}_i^{(-i)} = \hat{y}_i - \frac{h_{ii}}{1 - h_{ii}} (y_i - \hat{y}_i)$. Calculons l'erreur : $y_i - \hat{y}_i^{(-i)} = y_i - \hat{y}_i + \frac{h_{ii}}{1 - h_{ii}} (y_i - \hat{y}_i) = (y_i - \hat{y}_i) \left(1 + \frac{h_{ii}}{1 - h_{ii}}\right) = \frac{y_i - \hat{y}_i}{1 - h_{ii}}$. Cette formule fondamentale permet d'evaluer la generalisation du modele en un seul passage algorithmique !

Chapitre 2

Travaux pratiques et simulations algorithmiques

2.1 Objectif

Coder en Python pur la creation de la matrice de Vandermonde, la multiplication matricielle, et utiliser une methode d'elimination de Gauss pour resoudre les equations normales de la regression polynomiale.

2.2 Code Python et Validation Mathematique

```
1 def transpose(M):
2     return [[M[j][i] for j in range(len(M))] for i in range(len(M[0]))]
3
4 def matmul(A, B):
5     res = [[0]*len(B[0]) for _ in range(len(A))]
6     for i in range(len(A)):
7         for j in range(len(B[0])):
8             for k in range(len(B)):
9                 res[i][j] += A[i][k] * B[k][j]
10    return res
11
12 def gauss_jordan(A, b):
13     # Resolution de Ax = b par pivot de Gauss
14     n = len(A)
15     aug = [row[:] + [b[i][0]] for i, row in enumerate(A)]
16     for i in range(n):
17         pivot = aug[i][i]
18         for j in range(n + 1):
19             aug[i][j] /= pivot
20         for k in range(n):
21             if k != i:
22                 factor = aug[k][i]
23                 for j in range(n + 1):
24                     aug[k][j] -= factor * aug[i][j]
25     return [[aug[i][n]] for i in range(n)]
26
27 def poly_fit(X_data, Y_data, degree):
28     # 1. Matrice de Vandermonde
29     n = len(X_data)
30     V = [[(x**j) for j in range(degree + 1)] for x in X_data]
31     Y_col = [[y] for y in Y_data]
```

```

32
33     # 2. Equations normales : (V^T V) a = V^T Y
34     Vt = transpose(V)
35     VtV = matmul(Vt, V)
36     VtY = matmul(Vt, Y_col)
37
38     # 3. Resolution
39     a_col = gauss_jordan(VtV, VtY)
40     return [a[0] for a in a_col]
41
42 \section{Test Unitaire Mathematique}
43 X = [0.0, 1.0, 2.0]
44 Y = [1.0, 3.0, 7.0] # Fait par y = 1 + x + x^2
45 coeffs = poly_fit(X, Y, 2)
46 assert abs(coeffs[0] - 1.0) < 1e-6
47 assert abs(coeffs[1] - 1.0) < 1e-6
48 assert abs(coeffs[2] - 1.0) < 1e-6
49 print("TP1 Valide : Interpolation exacte reussie.")

```

2.3 Objectif

Demontrer empiriquement le phenomene de Runge. Interpoler une fonction sur un grand nombre de points reguliers et observer l'explosion de l'erreur aux bornes de l'intervalle avec l'augmentation du degre.

2.4 Code Python et Validation Mathematique

```

1 # Fonctions du TP1 (matmul, transpose, gauss_jordan, poly_fit) reprises
  implicitement
2 # Implementation allgee pour se concentrer sur l'algorithme
3
4 def runge_function(x):
5     return 1.0 / (1.0 + 25.0 * x**2)
6
7 def eval_poly(coeffs, x):
8     res = 0.0
9     for i, c in enumerate(coeffs):
10         res += c * (x**i)
11     return res
12
13 1. Generer points d'entrainement (equidistants)
14 degree = 10
15 n_train = degree + 1
16 X_train = [-1.0 + (2.0 * i) / degree for i in range(n_train)]
17 Y_train = [runge_function(x) for x in X_train]
18
19 2. Entrainer le polynome (avec regularisation infime pour stabilite)
20 (Supposons poly_fit_ridge implemente, similaire au TP1 avec + lambda
   sur diagonale)
21 def poly_fit_ridge(X_data, Y_data, degree, lam=1e-8):
22     V = [[(x**j) for j in range(degree + 1)] for x in X_data]
23     Vt = [[V[j][i] for j in range(len(V))] for i in range(len(V[0]))]
24     VtV = [[sum(a*b for a,b in zip(X_row, Y_col)) for Y_col in zip(*V)]
              for X_row in Vt]

```

```

25     VtY = [[sum(a*b for a,b in zip(X_row, [y])) for y in Y_data] for
26           X_row in Vt] # Simplified
27
28     # Real VtY
29     Y_col = [[y] for y in Y_data]
30     VtY = [[sum(Vt[i][k] * Y_col[k][0] for k in range(len(Y_col)))) for
31           i in range(len(Vt))]
32
33     for i in range(len(VtV)): VtV[i][i] += lam
34     # Simplified Gauss
35     aug = [row[:] + [VtY[i][0]] for i, row in enumerate(VtV)]
36     n = len(VtV)
37     for i in range(n):
38         aug[i] = [val / aug[i][i] for val in aug[i]]
39         for j in range(n):
40             if i != j:
41                 aug[j] = [aug[j][k] - aug[j][i]*aug[i][k] for k in
42                     range(n+1)]
43     return [aug[i][n] for i in range(n)]
44
45 coeffs = poly_fit_ride(X_train, Y_train, degree)
46
47 3. Test de l'erreur aux bords vs au centre
48 error_center = abs(runge_function(0.0) - eval_poly(coeffs, 0.0))
49 error_edge = abs(runge_function(0.95) - eval_poly(coeffs, 0.95))
50
51 assert error_edge > 5 * error_center, "L'erreur au bord doit exploser
52 par rapport au centre (Runge)."
53 print(f"TP2 Valide : Erreur centre = {error_center:.4f}, Erreur bord =
54 {error_edge:.4f}")

```

2.5 Objectif

Implementer la penalite L_2 au cœur des equations normales. Etudier son effet sur des donnees tres bruitees et comparer les coefficients avec et sans regularisation pour demontrer la reduction de variance (shrinkage).

2.6 Code Python et Validation Mathematique

```

1 def fit_ride(X_data, Y_data, degree, lambda_reg):
2     V = [[(x**j) for j in range(degree + 1)] for x in X_data]
3     Vt = [[V[j][i] for j in range(len(V))] for i in range(len(V[0]))]
4
5     VtV = [[sum(Vt[i][k] * V[k][j] for k in range(len(V))) for j in
6           range(len(V[0]))] for i in range(len(Vt))]
7
8     # Ajout de la penalite Ridge sur la diagonale (sauf biais)
9     for i in range(1, len(VtV)): # On ne regularise generalement pas le
10         terme constant a_0
11         VtV[i][i] += lambda_reg
12
13     Y_col = [[y] for y in Y_data]
14     VtY = [[sum(Vt[i][k] * Y_col[k][0] for k in range(len(Y_col)))) for
15           i in range(len(Vt))]

```

```

14     aug = [row[:] + [VtY[i][0]] for i, row in enumerate(VtV)]
15     n = len(VtV)
16     for i in range(n):
17         pivot = aug[i][i]
18         aug[i] = [val / pivot for val in aug[i]]
19         for j in range(n):
20             if i != j:
21                 aug[j] = [aug[j][k] - aug[j][i]*aug[i][k] for k in
22                             range(n+1)]
23     return [aug[i][n] for i in range(n)]
24
25 Donnees mal conditionnees (points tres proches)
26 X_data = [1.0, 1.01, 1.02]
27 Y_data = [2.0, 2.01, 2.02]
28
29 coeffs_no_reg = fit_ridge(X_data, Y_data, 2, 0.0)
30 coeffs_reg = fit_ridge(X_data, Y_data, 2, 1.0)
31
32 norm_no_reg = sum(c**2 for c in coeffs_no_reg[1:])
33 norm_reg = sum(c**2 for c in coeffs_reg[1:])
34
35 assert norm_reg < norm_no_reg, "La regularisation Ridge doit reduire la
    norme des coefficients (Shrinkage)."
36 print(f"TP3 Valide. Norme sans Ridge: {norm_no_reg:.2f} | Norme avec
    Ridge: {norm_reg:.2f}")

```

2.7 Objectif

Implementer la descente de gradient par lots (Batch Gradient Descent) pour optimiser les coefficients sans jamais inverser de matrice. Illustrer la notion de pas d'apprentissage (learning rate) et d'époques.

2.8 Code Python et Validation Mathématique

```

1 def eval_poly(coeffs, x):
2     return sum(c * (x**i) for i, c in enumerate(coeffs))
3
4 def gradient_descent_poly(X_data, Y_data, degree, epochs=1000, lr=0.01):
5     :
6     # Initialisation des coefficients a 0
7     coeffs = [0.0] * (degree + 1)
8     n = len(X_data)
9
10    for epoch in range(epochs):
11        # Calcul du gradient
12        grads = [0.0] * (degree + 1)
13        for i in range(n):
14            x = X_data[i]
15            y_pred = eval_poly(coeffs, x)
16            error = y_pred - Y_data[i]
17
18            # dLoss/dcoeff_k = 2 * error * (d_y_pred/dcoeff_k) = 2 *
19            # error * x^k
20            for k in range(degree + 1):
21                grads[k] += 2.0 * error * (x**k)

```

```

20
21     # Mise a jour
22     for k in range(degree + 1):
23         coeffs[k] -= lr * (grads[k] / n) # Moyenne du gradient
24
25     return coeffs
26
27 Test sur une droite
28 X = [0.0, 1.0, 2.0]
29 Y = [3.0, 5.0, 7.0] # y = 2x + 3
30
31 final_coeffs = gradient_descent_poly(X, Y, 1, epochs=2000, lr=0.1)
32
33 assert abs(final_coeffs[0] - 3.0) < 0.1, "Biais incorrect"
34 assert abs(final_coeffs[1] - 2.0) < 0.1, "Pente incorrecte"
35 print(f"TP4 Valide. Coefficients SGD trouves : {final_coeffs}")

```

2.9 Objectif

Ecrire l'algorithme complet de Validation Croisee Leave-One-Out (sans la formule analytique pour prouver le concept informatique) afin de determiner le degre optimal d'un polynome pour un jeu de donnees. Le but est de trouver le point d'equilibre entre biais et variance.

2.10 Code Python et Validation Mathematique

```

1 Utilisation de la fonction fit_ridge du TP3
2 def fit(X_data, Y_data, degree):
3     # Reprise du fit avec lambda tres faible
4     V = [[(x**j) for j in range(degree + 1)] for x in X_data]
5     Vt = [[V[j][i] for j in range(len(V))] for i in range(len(V[0]))]
6     VtV = [[sum(Vt[i][k] * V[k][j] for k in range(len(V))) for j in
7             range(len(V[0]))] for i in range(len(Vt))]
8     for i in range(len(VtV)): VtV[i][i] += 1e-9
9     Y_col = [[y] for y in Y_data]
10    VtY = [[sum(Vt[i][k] * Y_col[k][0] for k in range(len(Y_col)))) for
11            i in range(len(Vt))]
12    aug = [row[:] + [VtY[i][0]] for i, row in enumerate(VtV)]
13    n = len(VtV)
14    for i in range(n):
15        aug[i] = [val / aug[i][i] for val in aug[i]]
16        for j in range(n):
17            if i != j:
18                aug[j] = [aug[j][k] - aug[j][i]*aug[i][k] for k in
19                          range(n+1)]
20    return [aug[i][n] for i in range(n)]
21
22 def eval_p(coeffs, x):
23     return sum(c * (x**i) for i, c in enumerate(coeffs))
24
25 def loocv_error(X, Y, degree):
26     n = len(X)
27     total_error = 0.0
28     for i in range(n):
29         # Retrait de la i-eme donnee
30         X_train = X[:i] + X[i+1:]

```

```

28     Y_train = Y[:i] + Y[i+1:]
29
30     # Entraînement
31     coeffs = fit(X_train, Y_train, degree)
32
33     # Test sur la donnée laissée de côté
34     prediction = eval_p(coeffs, X[i])
35     total_error += (prediction - Y[i])**2
36
37     return total_error / n
38
39 Données générées par une parabole bruitée
40 X_data = [-2.0, -1.0, 0.0, 1.0, 2.0, 3.0]
41 y = x^2 avec un peu de bruit
42 Y_data = [4.1, 1.0, 0.1, 1.1, 3.9, 9.2]
43
44 Testons les degrés 1 à 4
45 errors = []
46 for d in range(1, 5):
47     err = loocv_error(X_data, Y_data, d)
48     errors.append(err)
49
50 optimal_degree = errors.index(min(errors)) + 1
51
52 Mathématiquement, le modèle sous-jacent est de degré 2. Le LOOCV doit
    le trouver.
53 assert optimal_degree == 2, f"Le LOOCV a échoué à identifier le vrai
    degré. Trouve: {optimal_degree}"
54 print(f"TP5 Valide. Erreurs LOOCV : {errors}")
55 print(f"Degré optimal sélectionné : {optimal_degree}")

```